上海电力大学 人工智能学部

操作系统原理 · 实验二

Linux 进程管理

进程观察、控制、调度与运行验证

陈心怡　杨骅　李金盘　李玥辰

数据科学与大数据技术 2024371 班

六个章节对应六项实验能力

3.1	进程观察与管理	进程列表、状态、资源和父子层级
3.2	系统调用与生命周期	创建、等待、回收、僵尸与守护进程
3.3	进程状态转换	PCB 、就绪队列、阻塞队列
3.4	三种调度算法	FCFS 、 SJF 、 RR 的队列规则
3.5	多进程实验与定量对照	事件轨迹、经历时间和 nice 对照
3.6	openEuler 环境搭建	虚拟机安装及系统服务验证

进程身份与父子关系

环境确认

uname 识别内核， os-release 识别发行版

身份字段

PID 标识进程， PPID 指向父进程

同名实例

相同 CMD 可以对应多个不同 PID

```
Linux process experiment — independent Debian 13 container
实验二：ps -ef 查看进程
------------------------------------------------------------

$ ps -ef | cut -c 1-112 | head -n 17
UID          PID    PPID  C STIME TTY          TIME CMD
root           1       0  0 12:47 ?        00:00:09 /usr/bin/python3 /usr/bin/supervisord -n -c /etc/supervisord
root           6       1  0 12:47 ?        00:00:00 log_forwarder --service-name logrotate_loop -- /bin/bash /us
root          55       6  0 12:47 ?        00:00:00 /bin/bash /usr/local/init_scripts/logrotate_loop.sh
root          78      55  0 12:47 ?        00:00:00 sleep 86400
root         197       1  0 12:47 ?        00:00:00 log_forwarder --service-name python_tool -- /bin/bash /usr/l
root         225       1  0 12:47 ?        00:00:00 log_forwarder --service-name terminal_server -- /bin/bash /u
root         240     225  0 12:47 ?        00:00:00 /bin/bash /usr/local/init_scripts/terminal_server.sh
oai          241     240  0 12:47 ?        00:00:00 /bin/bash /opt/terminal-server/scripts/start-server.sh
root         247     197  0 12:47 ?        00:00:00 /bin/bash /usr/local/init_scripts/python_tool.sh
oai          258     247  0 12:47 ?        00:00:01 tini -- /opt/pyvenv-python-tool/bin/python -m uvicorn --host
oai          278     241  4 12:47 ?        00:00:40 /opt/terminal-server/pyvenv/bin/python /opt/terminal-server/
oai          296     258  7 12:47 ?        00:01:09 /opt/pyvenv-python-tool/bin/python -m uvicorn --host 0.0.0.0
oai          325     296  1 12:47 ?        00:00:19 /opt/pyvenv/bin/python -m ipykernel_launcher -f /tmp/tmphkw4
oai          344     325  4 12:47 ?        00:00:39 /opt/pyvenv/lib/python3.13/site-packages/artifact_tool/bin/a
root        1571       0  0 13:03 ?        00:00:00 timeout 35s bash -lc python - <<'PY' p='/mnt/data/capture_ch
root        1572    1571  0 13:03 ?        00:00:00 bash -lc python - <<'PY' p='/mnt/data/capture_ch1.py' s=open

# All displayed outputs above were produced by commands in this session.
```

报告图 4： ps -ef 的实际输出

```
ps -ef
ps -eo pid,ppid,stat,ni,%cpu,%mem,comm
```

先确认实验环境，再用 PID / PPID 识别进程及其创建关系

状态与资源： ps 快照和 top 采样

字段	含义
R	运行或可运行
S	可中断睡眠
Z	退出但尚未回收
NI	nice 值
%CPU / %MEM	资源指标

```
Linux process experiment — independent Debian 13 container
实验三：ps 自定义字段与进程状态
------------------------------------------------------------

$ ps -eo pid,ppid,stat,ni,%cpu,%mem,comm --sort=pid | head -n 25
    PID    PPID STAT  NI %CPU %MEM COMMAND
      1       0 Ss     0  0.9  0.4 supervisord
      6       1 Sl     0  0.0  0.0 log_forwarder
     55       6 S      0  0.0  0.0 bash
     78      55 S      0  0.0  0.0 sleep
    197       1 Sl     0  0.0  0.0 log_forwarder
    225       1 Sl     0  0.0  0.0 log_forwarder
    240     225 S      0  0.0  0.0 bash
    241     240 S      0  0.0  0.0 start-server.sh
    247     197 S      0  0.0  0.0 bash
    258     247 Sl     0  0.1  0.0 tini
    278     241 S      0  4.0  0.6 python
    296     258 Sl     0  7.0  1.2 python
    325     296 Ssl    0  1.9  2.3 python
    344     325 Ssl    0  4.0  4.1 artifact_tool_r
   1571       0 Ss     0  0.0  0.0 timeout
   1572    1571 S      0  0.0  0.0 bash
   1586    1572 Sl     0 52.0  1.7 python
   1596    1586 S      0 15.3  0.6 Xvfb
```

ps： 按进程读取状态与资源字段

```
Linux process experiment — independent Debian 13 container
实验四：top 动态进程监控（单次采样）
------------------------------------------------------------

$ top -b -n 1 | head -n 24
top - 13:04:05 up 16 min,  0 users,  load average: 0.28, 0.34, 0.27
Tasks:  23 total,   1 running,  22 sleeping,   0 stopped,   0 zombie
%Cpu(s):  1.9 us,  0.0 sy,  0.0 ni, 96.2 id,  0.0 wa,  0.0 hi,  0.0 si,  1.9 st
MiB Mem :   5959.2 total,   4814.3 free,   1248.4 used,     72.7 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.   4710.8 avail Mem

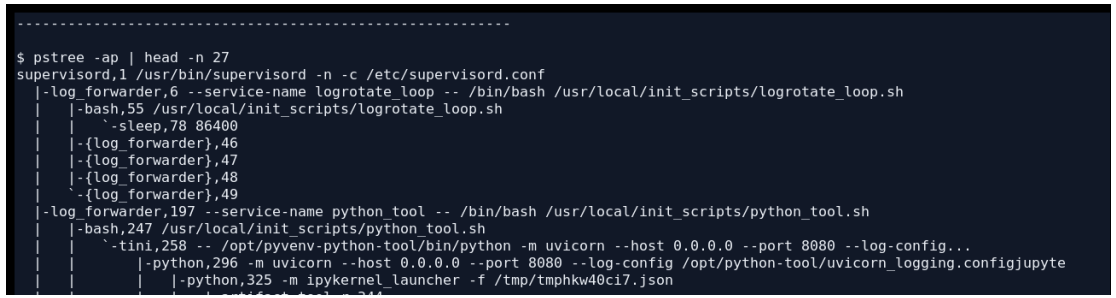
    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0   39704  26256   6908 S   0.0   0.4   0:09.46 supervi+
      6 root      20   0  275548   2000   1632 S   0.0   0.0   0:00.02 log_for+
     55 root      20   0    4456   1840   1528 S   0.0   0.0   0:00.00 bash
```

top： 23 total， 1 running， 22 sleeping

S 表示睡眠等待； ps 与 top 都只反映各自采样时刻

父子层级与临时进程的退出

父子关系



报告图 7： pstree 的局部输出

```
pstree -ap
```

创建、观察、终止与回收



报告图 8： sleep 测试进程的退出

```
sleep 90 & demo_pid=$!
kill "$demo_pid"; wait "$demo_pid"
```

进程管理既要观察结构，也要确认测试进程完成退出与回收

getpid / getppid： 实例变化，父进程保持一致

```
printf("PID=%d\n", getpid());
printf("PPID=%d\n", getppid());
```

运行次数	PID	PPID
第一次	2108	327
第二次	13097	327
第三次	13107	327

```
yh180@Lenovo:~/os_lab$ gcc p21.c -o p21
yh180@Lenovo:~/os_lab$ ./p21
此进程的进程号(PID)是： 2108
此进程的父进程号(PPID)是： 327
```

第一次运行

```
yh180@Lenovo:~/os_lab$ ./p21
此进程的进程号(PID)是： 13097
此进程的父进程号(PPID)是： 327
```

第二次运行

```
此进程的进程号(PID)是： 13107
此进程的父进程号(PPID)是： 327
yh180@Lenovo:~/os_lab$
```

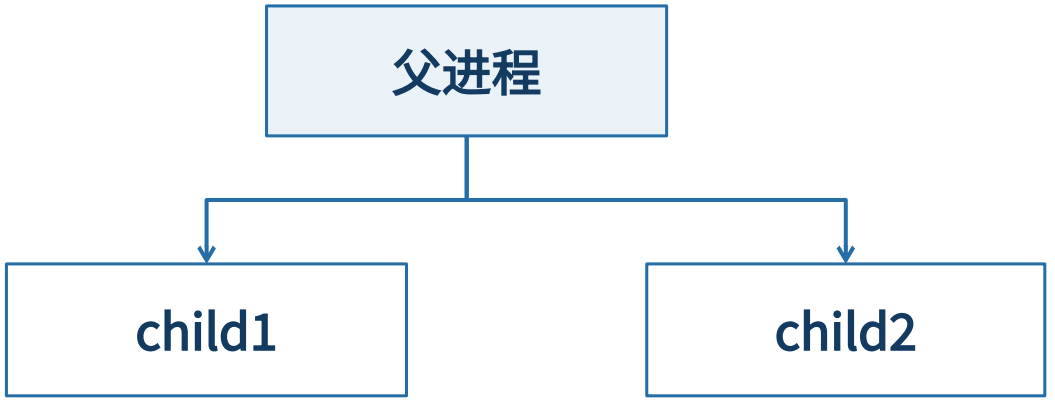
第三次运行

PID 对应执行实例；三次 PPID=327 对应同一个启动它们的 Shell

fork： 父进程连续创建两个子进程

关键分支摘录

```
pid1 = fork();
if (pid1 == 0) {
    printf("child1\n");
} else {
    pid2 = fork();
    if (pid2 == 0)
        printf("child2\n");
    else
        printf("parent\n");
}
```




```
yh180@Lenovo:~/os_lab$ gcc p22.c -o p22
yh180@Lenovo:~/os_lab$ ./p22
child1
parent
child2
```

报告图 17： 本次输出顺序

第二次 fork 位于父分支，因此是一个父进程与两个直接子进程

两个子进程的身份关系与执行分支

角色	PID	PPID
父进程	13451	—
子进程 A	13452	13451
子进程 B	13453	13451

```
/*  子进程完成自己的分支  */
exit(0);
/*  父进程回收两个子进程  */
wait(NULL);
wait(NULL);
```

```
yh180@Lenovo:~/os_lab$ ./p22
child1
parent
child2
yh180@Lenovo:~/os_lab$ ./p22
child1
parent
child2
yh180@Lenovo:~/os_lab$ ./p22
child1
parent
child2
yh180@Lenovo:~/os_lab$
yh180@Lenovo:~/os_lab$ ./p22
child1
parent
child2
```

报告图 18： 多次运行结果

创建关系由 PID / PPID 验证； exit 与 wait 分别结束子分支和完成回收

僵尸进程：子进程已退出，父进程尚未回收

```
pc = fork();
if (pc == 0) {
    printf("child PID=%d\n",
           getpid());
} else {
    sleep(20);
    /* 此处没有调用 wait */
}
exit(0);
```

观察窗口

在父进程 20 秒休眠期间查询子进程

```
yh180@Lenovo:~/os_lab$ ./p24 &
[1] 13762
This is child process with pid of 13765
yh180@Lenovo:~/os_lab$ ps -o pid,ppid,stat,cmd -p 13756
    PID    PPID STAT CMD
  13756   13755 Z+   [p24] <defunct>
yh180@Lenovo:~/os_lab$
```

报告图 24： ps 查询到 PID 13756 为 Z+

Z+

退出状态仍等待父进程读取

截图新建 PID 与 ps 查询 PID 不同，需复核对应关系

僵尸状态的关键是“已经退出但尚未回收”

wait： 等待结束并读取退出状态

```
/*  子进程  */
sleep(5);
exit(5);

/*  父进程  */
wpid = wait(&status);
if (wpid > 0 && WIFEXITED(status))
    printf("status=%d\n",
           WEXITSTATUS(status));
```

```
yh180@Lenovo:~/os_lab$ nano p25.c
yh180@Lenovo:~/os_lab$ gcc p25.c -o p25
yh180@Lenovo:~/os_lab$ ./p25
Parent process waiting for child...
Child process. PID = 13842
Waited child PID = 13842
Child exit status = 5
yh180@Lenovo:~/os_lab$
```

报告图 26： 回收 PID 13842 ， 退出码为 5

返回信息	本次结果
wait 返回的 PID	13842
WEXITSTATUS	5

wait 同时承担父子进程同步与终止状态回收

守护进程：初始化与业务逻辑分离



可复用初始化

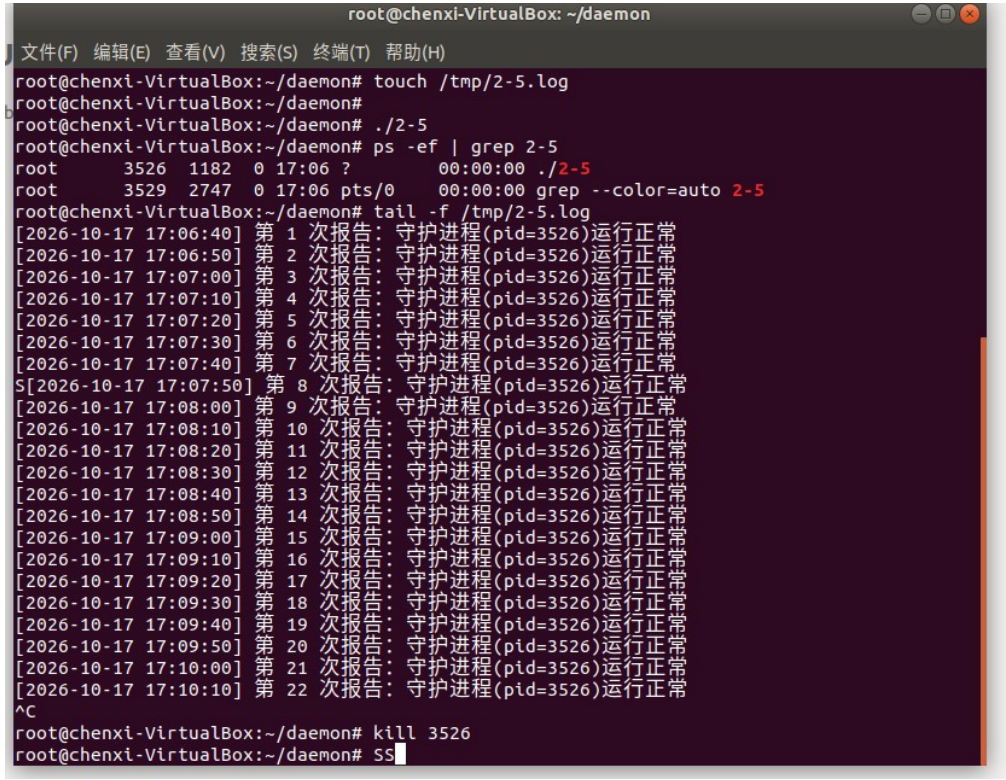
```
init.c
init_daemon(0);
```

独立业务程序

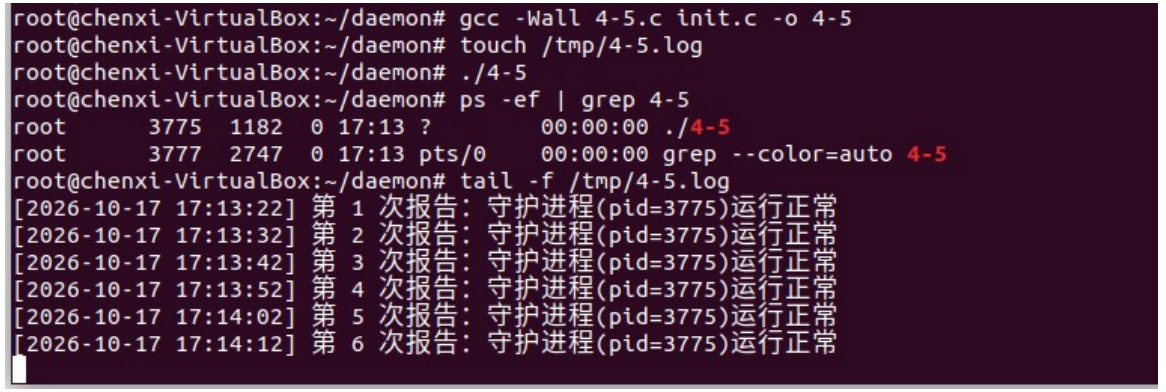
```
2-5.c / 4-5.c
fopen(path, "a");
fprintf(...); fclose(...);
sleep(10);
```

初始化清理继承环境；业务程序通过日志报告周期任务状态

守护进程验证：进程状态与周期日志



任务 1： PID 3526， 22 条记录



任务 2： PID 3775， 6 条记录

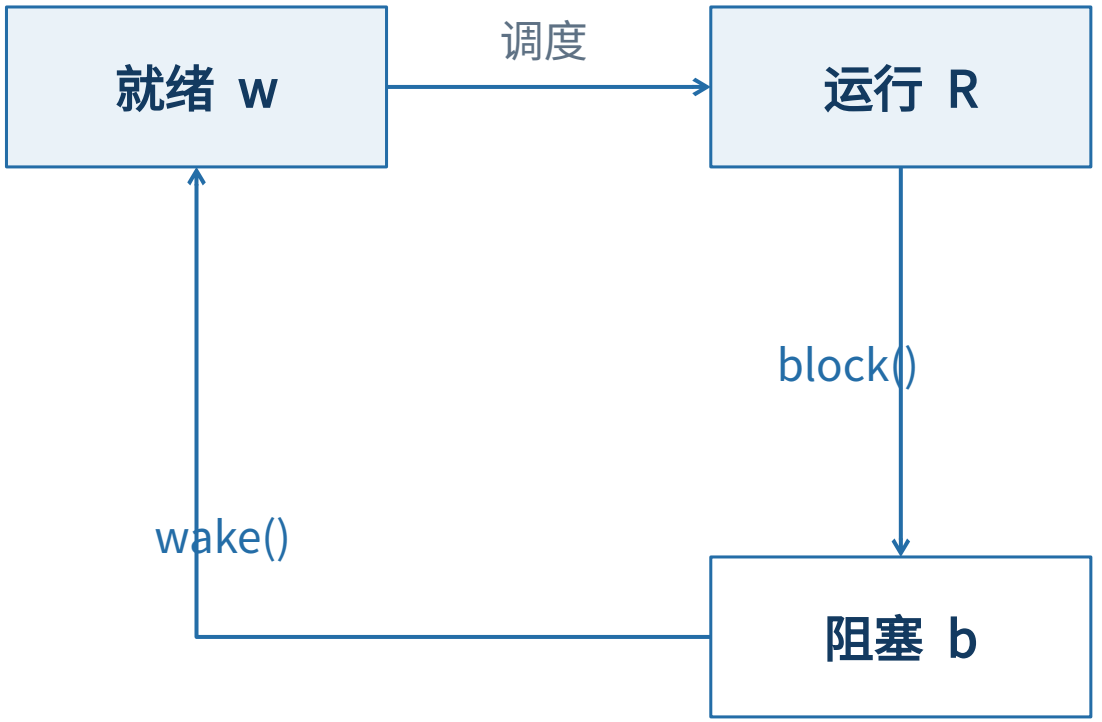
核验项目	任务 1	任务 2
首尾跨度	210 s	50 s
日志间隔数	21	5
相邻间隔	10 s	10 s

TTY 与稳定 PID 验证后台运行；日志时间差验证观测窗口内的周期性

PCB 与就绪、阻塞两条队列

PCB 字段	作用
name / state	名称与状态
ntime / rtime	所需与已运行时间
link	同队列的下一个 PCB

ready / pfend： 就绪队首、队尾
stop / stepend： 阻塞队首、队尾



状态转换需要同时改变 state 和进程所在链表

block / wake 的核心是摘链与挂链

block： 阻塞队列非空时的分支

```
ready->state = 'b';
p = ready;
ready = ready->link;
p->link = NOTHING;
stepend->link = p;
stepend = p;
```

wake： 就绪队列非空时的分支

```
p = stop;
stop = stop->link;
p->state = 'w';
p->link = NOTHING;
pfend->link = p;
pfend = p;
```

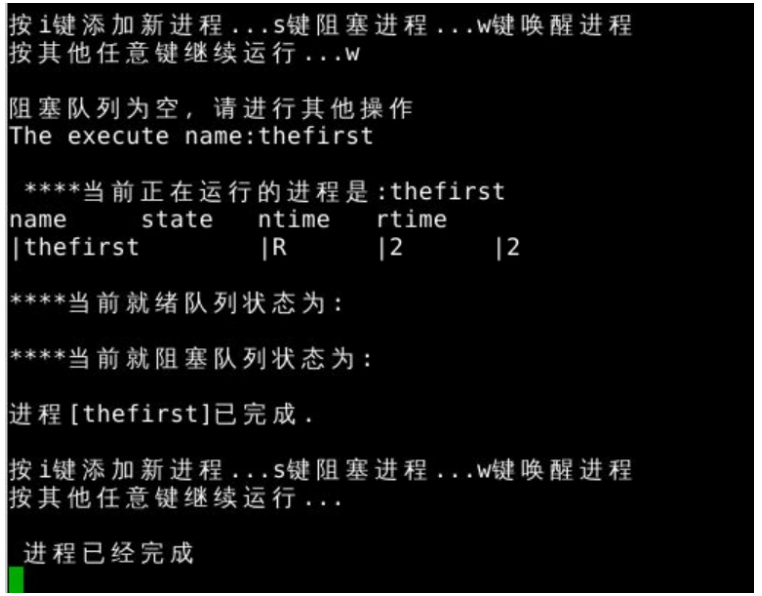
空队列、单结点和尾指针需要同步处理；唤醒前先检查阻塞队列是否为空。

实现顺序：保存结点、移动队首、断开旧链、接入新队列

三进程实验的状态轨迹与边界验证

阶段	运行	就绪	阻塞
初始	A 1/2	B 、 C	空
按 s 后	B 完成， C 运行	空	A
按 w 后	C 2/3	A	空
C 完成	A 2/2	空	空
空队列再按 w	提示，不改变队列	空	空

A=thefirst B=thesecond C=thethird



报告图 37： 空队列提示及全部完成

唤醒让进程回到就绪队尾；是否立即运行由当前调度规则决定

FCFS： 新进程进入队尾

插入规则

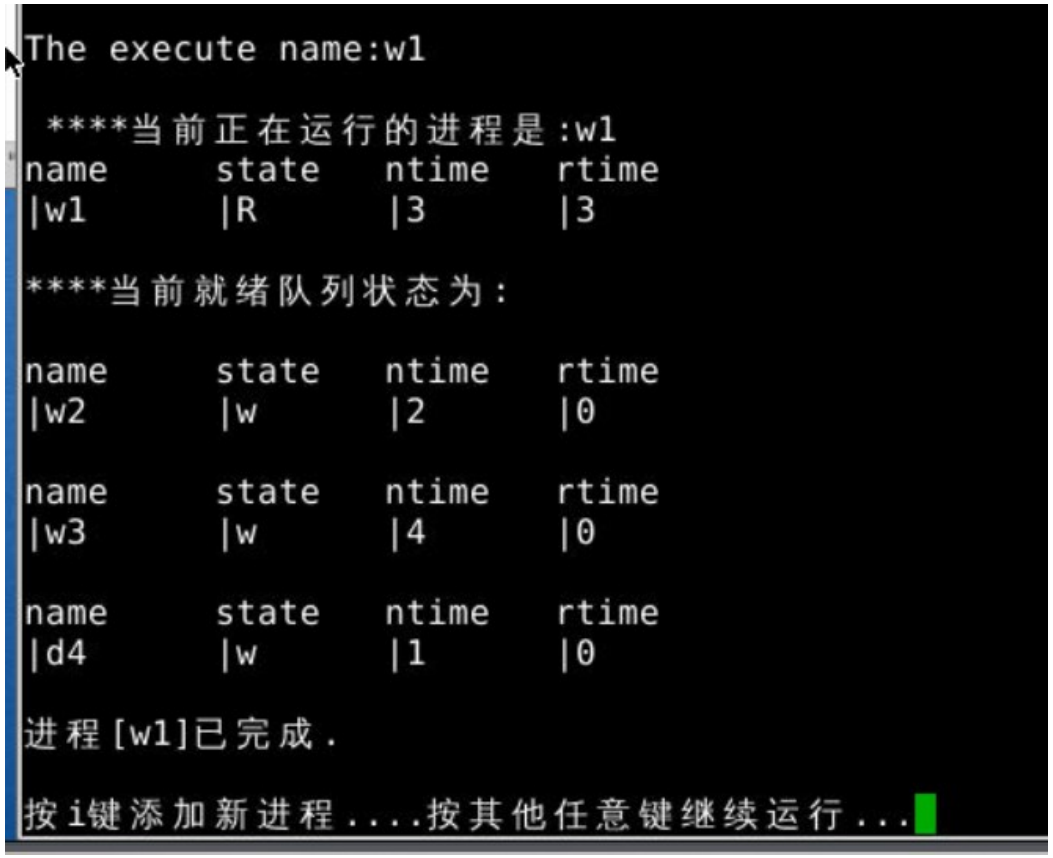
沿 link 到达队尾，把新 PCB 接在最后

运行规则

当前进程每次运行一个时间单位

完成判断

rtime 等于 ntime 后撤销，再运行下一项



报告图 41： w1 完成前的就绪队列

FCFS 保持到达顺序；后到的短进程也需要等待

SJF： 等待队列按所需运行时间排序

比较依据

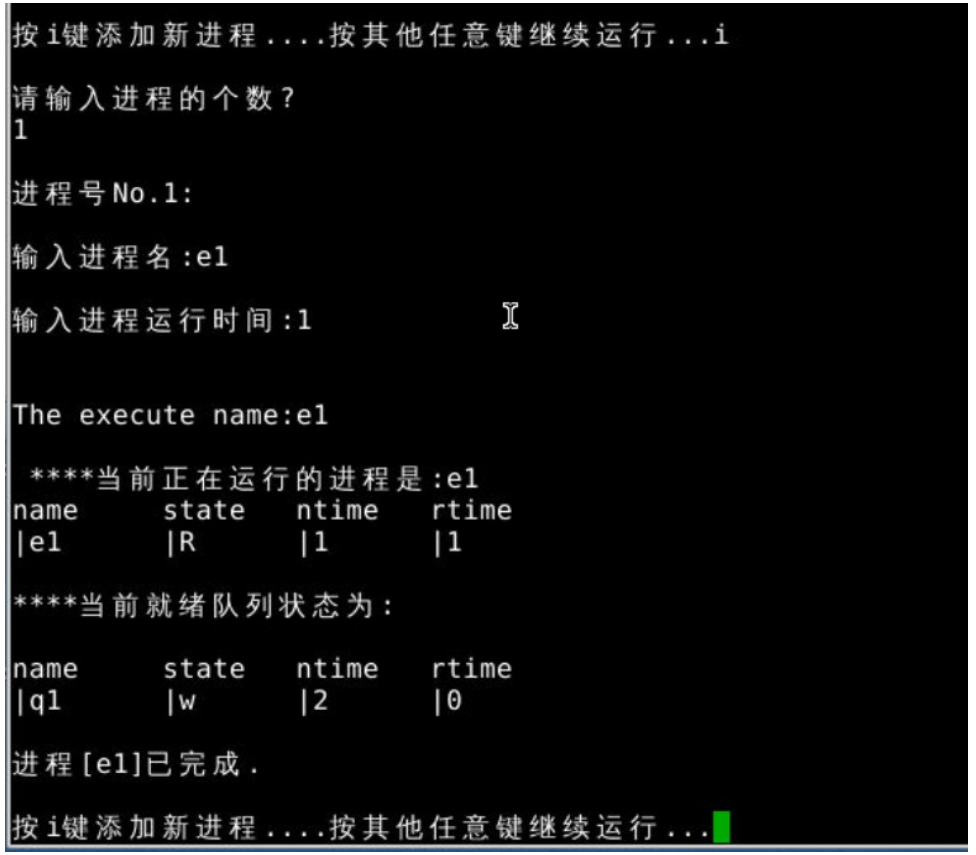
按预计需要运行时间 ntime 插入

非抢占约束

保留已开始运行的队首进程

主要局限

持续到来的短任务可能使长任务等待



报告图 47： 短进程 e1 的运行结果

短任务优先选择与当前任务是否可被抢占，是两个不同的实现条件

RR：时间片用尽后回到队尾

运行阶段	执行者	行为
前两个时间单位	r1	rtime 到 2，余量到 0
下一时间单位	r2	运行后完成
后续时间单位	r1	继续到 rtime=3

未完成：放回队尾
已完成：撤销并移出队列

```
The execute name:r1

 ****当前正在运行的进程是:r1
name      state   ntime   rtime   ltime
|r1       |R      |3      |2      |0

****当前就绪队列状态为:

name      state   ntime   rtime   ltime
|r2       |w      |1      |0      |2

按i键添加新进程....按其他任意键继续运行...

The execute name:r2

 ****当前正在运行的进程是:r2
name      state   ntime   rtime   ltime
|r2       |R      |1      |1      |1

****当前就绪队列状态为:

name      state   ntime   rtime   ltime
|r1       |w      |3      |2      |2

进程[r2]已完成.
```

报告图 50 局部：时间片字段显示为 ltime

轮转依靠时间片触发队列调整，避免单个未完成任务一直占用处理机

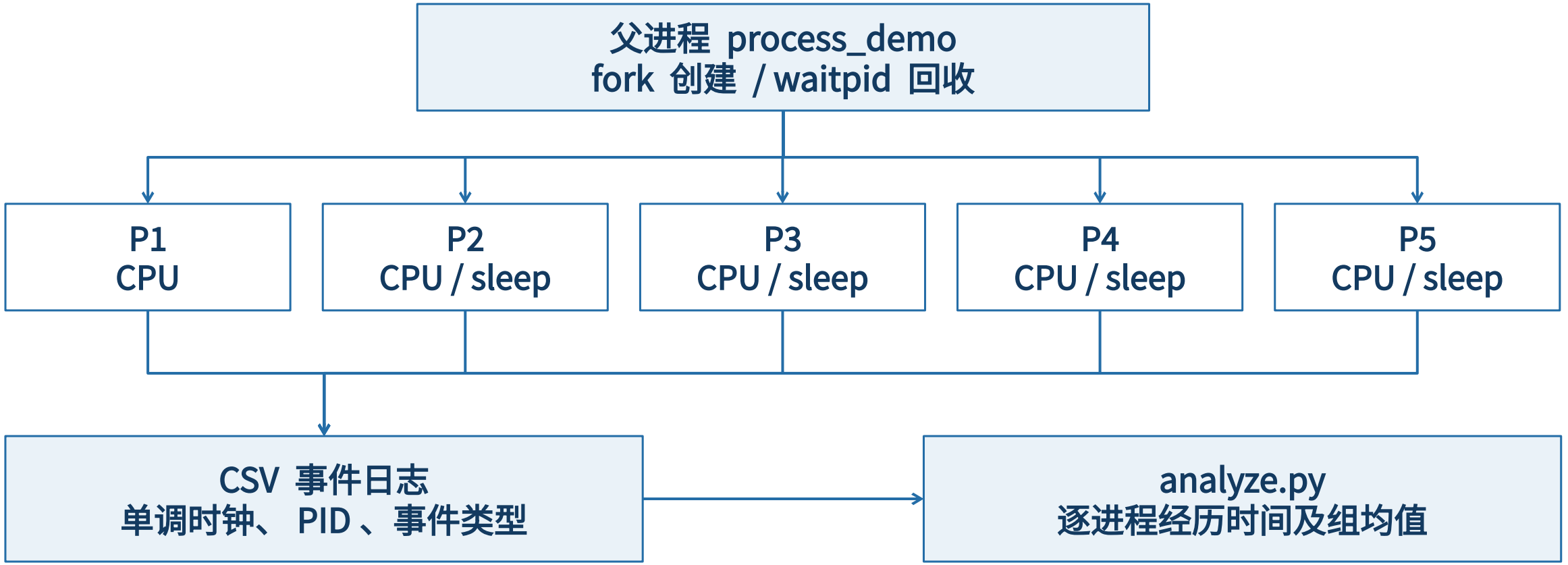
三种调度算法的规则对比

比较维度	FCFS	非抢占 SJF	RR
选择依据	到达顺序	预计运行时间	轮转顺序
是否抢占	否	否	时间片用尽时轮转
核心队列操作	新进程接队尾	按 ntime 排序插入	未完成者移队尾
主要特点	实现简单	有利于短任务等待	改善交互响应
主要局限	短任务可能久等	长任务可能饥饿	受时间片长度影响

本节验证调度规则。报告未提供同一工作负载下的三算法定量指标。

定量评价还需要统一工作负载、统计口径与重复测量

五子进程实验的技术架构



事件采集与统计分析相连，进程创建和回收由父进程统一完成

编译、自检与对照条件

```
gcc -std=c11 -O2 -Wall -Wextra \
    -Werror process_demo.c -o process_demo
./process_demo --check
```



报告图 52 局部：自检与五组负载

控制条件	设置
子进程数	5
CPU 亲和性	taskset -c 0
两组 nice 值	0 / 10
计算与休眠负载	保持一致

先验证负载定义，再在相同条件下比较不同 nice 参数

进程创建与状态观测

```
第五章 / Linux 进程调度实验      五个子进程创建与状态
运行环境: Debian GNU/Linux 13 / x86_64
--------------------------------------------------------------------
$ ./process_demo observed.csv > observed.txt &
$ cat observed.txt
HOST Linux demo, five children, parent_pid=4456, nice=0
CREATED P1 pid=4458
CREATED P2 pid=4459
CREATED P3 pid=4460
CREATED P4 pid=4461
CREATED P5 pid=4462
$ ps -o pid,ppid,stat,ni,%cpu,comm --ppid <父进程PID>
    PID    PPID STAT  NI %CPU COMMAND
   4458    4456 R+     0 96.2 process_demo
   4459    4456 S+     0 16.9 process_demo
   4460    4456 S+     0 83.0 process_demo
   4461    4456 S+     0 28.3 process_demo
   4462    4456 R+     0 86.7 process_demo
本次父进程 PID=4456
```

报告图 53 局部：父子进程及状态

父进程

PID 4456

五个子进程

PID 4458—4462

状态含义

R+： 运行或可运行

S+： 可中断睡眠

PID / PPID 证明创建关系， R / S 说明样本包含不同运行阶段

事件轨迹与经历时间的统计口径

事件类别	记录用途
START / EXIT	进程生命周期起止
CPU_BEGIN / CPU_END	计算阶段边界
IO_SLEEP_BEGIN / END	休眠阶段边界

T 经历 = tEXIT − tSTART

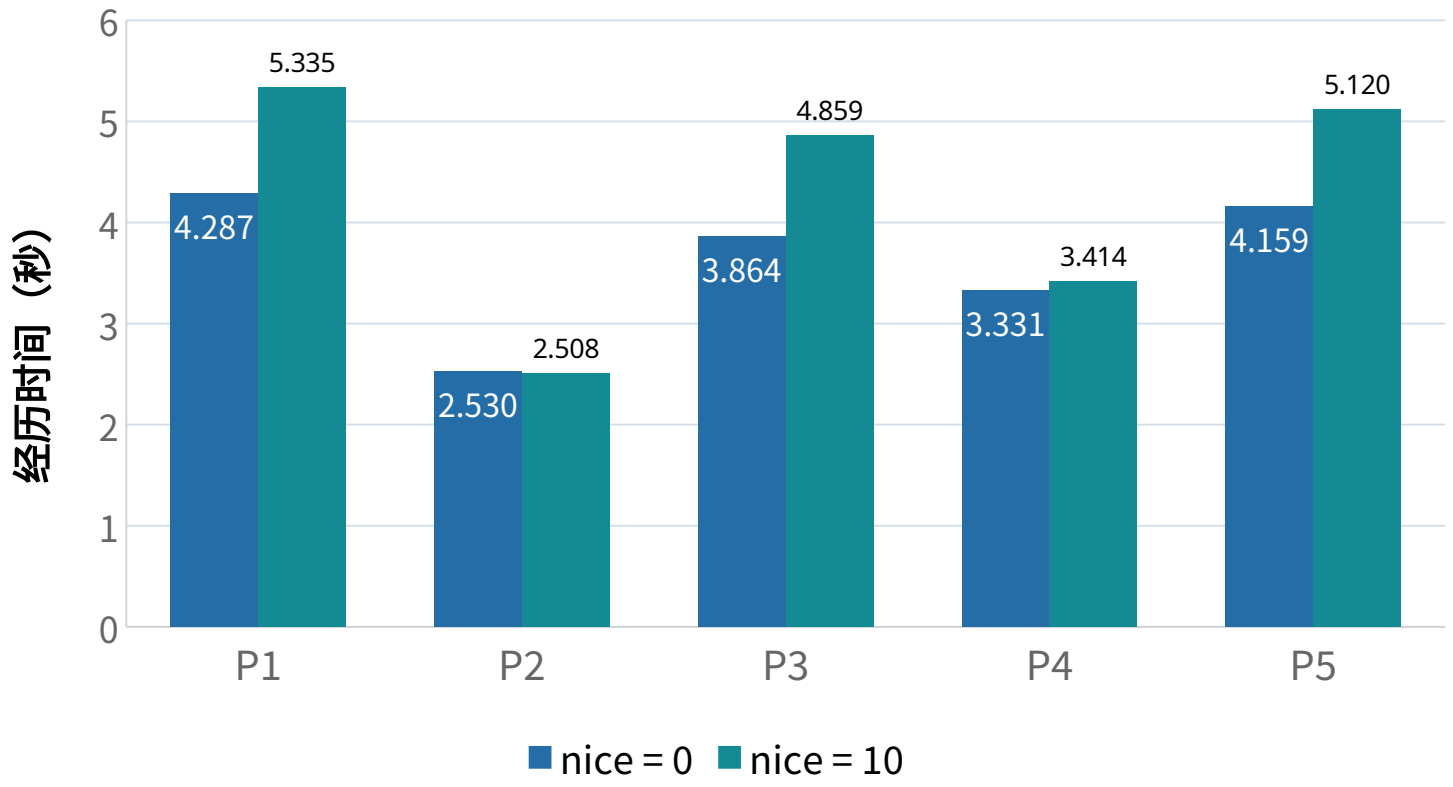
包含计算、休眠和调度等待

```
$ head -n 12 observed.csv
t_seconds,child,pid,event
0.004,2,4459,START
0.005,2,4459,CPU_BEGIN
0.006,4,4461,START
0.006,4,4461,CPU_BEGIN
0.008,3,4460,START
0.008,1,4458,START
0.008,3,4460,CPU_BEGIN
0.008,1,4458,CPU_BEGIN
0.012,5,4462,START
0.012,5,4462,CPU_BEGIN
0.112,2,4459,CPU_END
$ tail -n 8 observed.csv
1.733,3,4460,IO_SLEEP_END
1.733,3,4460,EXIT
1.741,5,4462,IO_SLEEP_END
1.741,5,4462,EXIT
1.816,2,4459,IO_SLEEP_END
1.816,2,4459,EXIT
2.408,4,4461,IO_SLEEP_END
2.408,4,4461,EXIT
```

报告图 54 局部： CSV 事件记录

比较前统一指标定义，避免把经历时间误当作 CPU 执行时间

nice 两组实验的经历时间对比



五进程均值

3.634 s

nice = 0

4.247 s

nice = 10

+0.613 s　约 +16.9%

本轮均值增加， P2 差异较小；结论限于当前负载和单轮测量

原始统计与运行完整性核验

```
第五章 / Linux 进程调度实验      统计结果对比
运行环境：Debian GNU/Linux 13 / x86_64
------------------------------------------------------------------------
$ python3 analyze.py actual_nice0.csv
P1 pid=3783 wall=4.287s CPUbursts=2 sleepbursts=0
P2 pid=3784 wall=2.530s CPUbursts=2 sleepbursts=2
P3 pid=3785 wall=3.864s CPUbursts=2 sleepbursts=2
P4 pid=3786 wall=3.331s CPUbursts=2 sleepbursts=2
P5 pid=3787 wall=4.159s CPUbursts=2 sleepbursts=2
Mean child wall time: 3.634s
$ python3 analyze.py actual_nice10.csv
P1 pid=3801 wall=5.335s CPUbursts=2 sleepbursts=0
P2 pid=3802 wall=2.508s CPUbursts=2 sleepbursts=2
P3 pid=3803 wall=4.859s CPUbursts=2 sleepbursts=2
P4 pid=3804 wall=3.414s CPUbursts=2 sleepbursts=2
P5 pid=3805 wall=5.120s CPUbursts=2 sleepbursts=2
Mean child wall time: 4.247s
```

报告图 57 局部：逐进程结果与均值

```
第五章 / Linux 进程调度实验      轨迹文件与完整性验证
运行环境：Debian GNU/Linux 13 / x86_64
------------------------------------------------------------------------
$ wc -l actual_nice0.csv actual_nice10.csv
  47 actual_nice0.csv
  47 actual_nice10.csv
  94 total
$ python3 -m py_compile analyze.py
静态检查状态：0
$ ls -lh actual_nice*.csv
-rw-r--r-- 1 root oai_shared 1.1K Oct 22 13:39 actual_nice0.csv
-rw-r--r-- 1 root oai_shared 1.1K Oct 22 13:39 actual_nice10.csv
$ grep -c REAPED actual_nice0.txt actual_nice10.txt
actual_nice0.txt:6
```

报告图 58 局部：日志、脚本与退出检查

日志完整性

两组 CSV 各 47 行，包含表头

运行结束

五个子进程均完成并回收

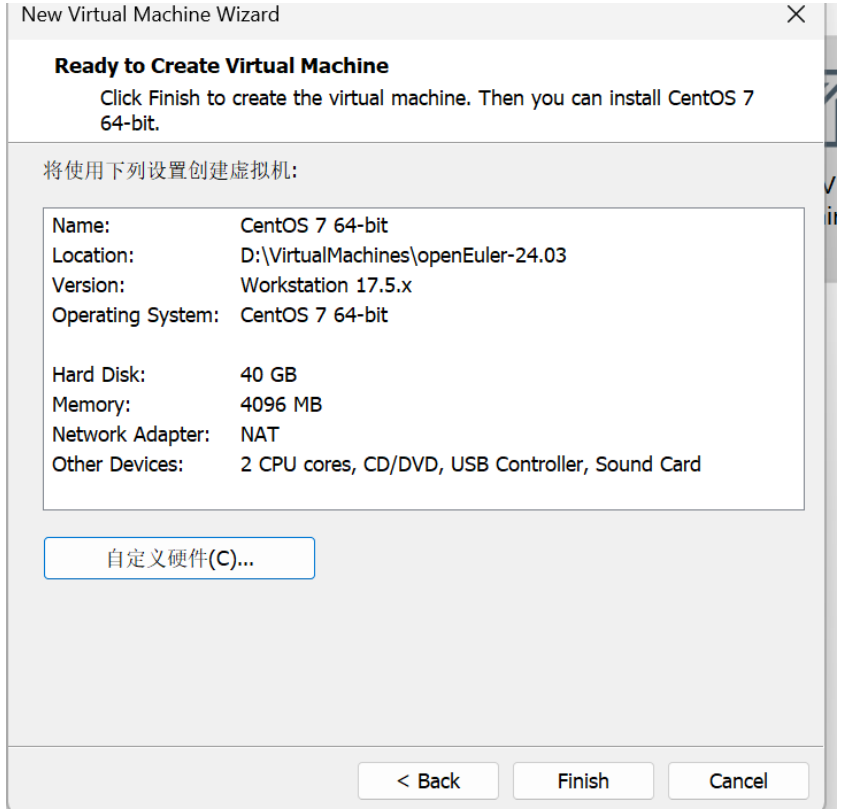
后续改进

重复多轮，控制竞争任务
统计均值与离散程度

运行完整性支持本次记录可用；稳定性能结论仍需重复实验

镜像与虚拟机配置

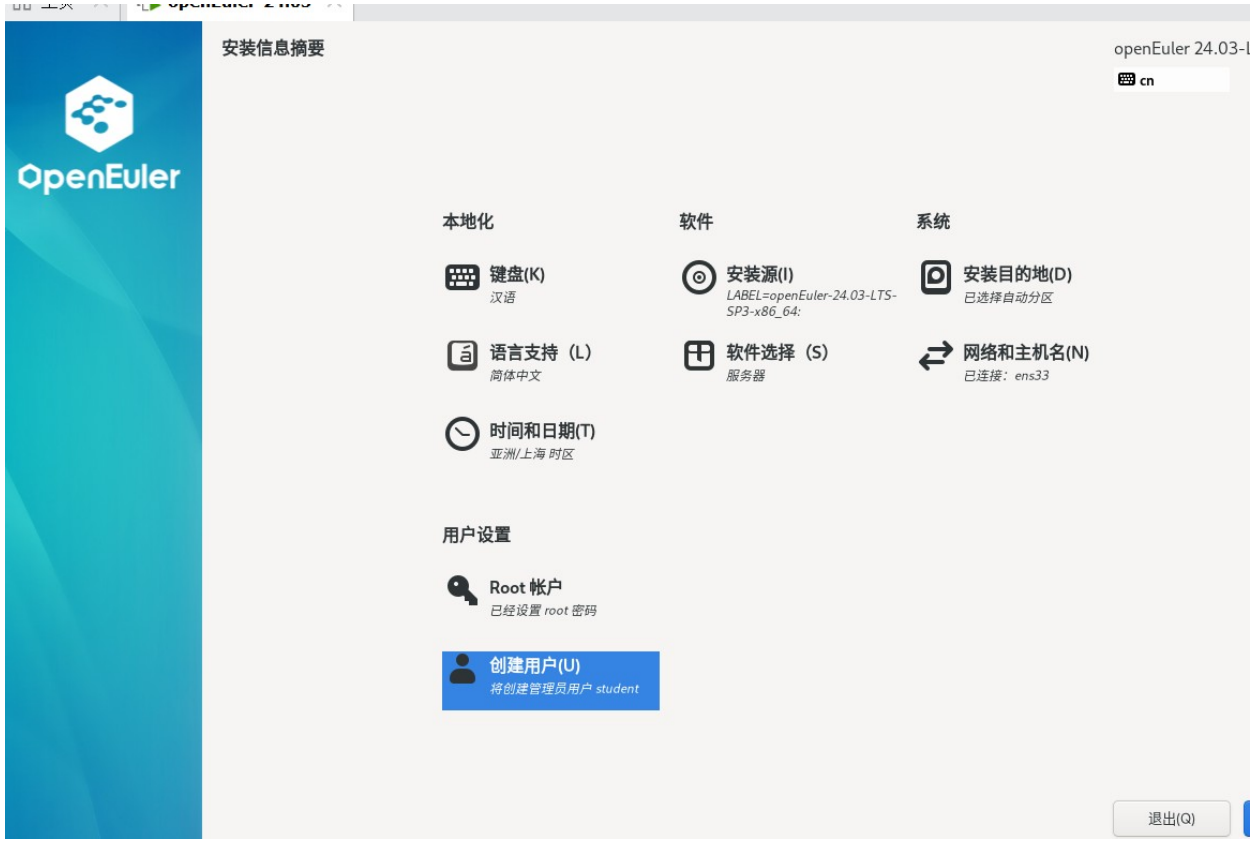
配置项	本次设置
系统镜像	openEuler 24.03 LTS SP3
体系结构	x86_64
CPU / 内存	2 核 / 4096 MB
磁盘 / 网络	40 GB / NAT
虚拟机软件	VMware Workstation 17.5.x



报告图 61： 虚拟机配置摘要

实验完成 x86_64 虚拟机安装，为后续系统实验提供环境

安装流程与用户配置



报告图 62： 安装信息摘要

安装设置

中文， Asia/Shanghai ，本地 ISO

磁盘与软件

自动分区，服务器软件环境

用户与启动

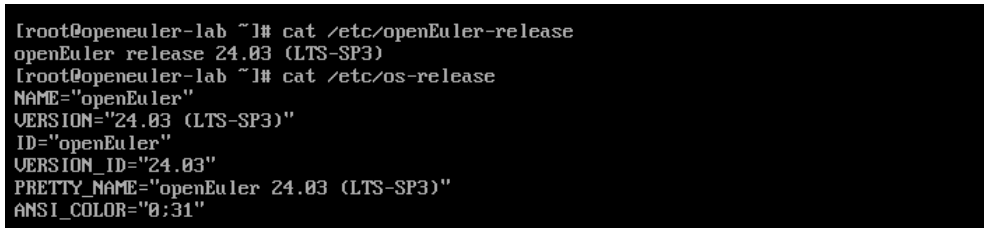
配置 root 和 student
安装完成后重启进入命令行

镜像、分区、网络与用户设置完成后，进入首次启动验证

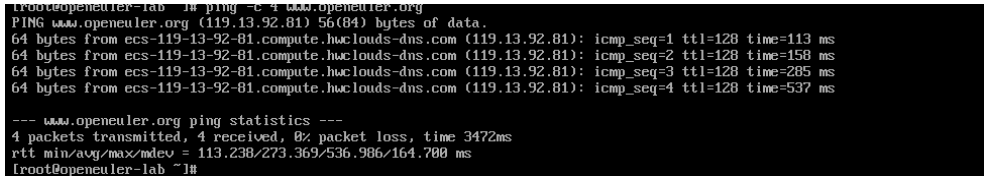
系统版本、地址与外网连通性

验证对象	本次结果
发行版	openEuler 24.03 LTS-SP3
网卡与地址	ens33 192.168.230.128/24
网络连通	4 发 / 4 收 / 0% 丢包

```
cat /etc/openEuler-release
ip addr
ping -c 4 www.openeuler.org
```



报告图 63 局部：系统版本信息



报告图 63 局部：外网探测结果

发行版以系统文件验证，网络以地址与收发结果验证

工具、 SSH 与防火墙验证

```
Complete!
[root@openeuler-lab ~]# sudo duf install -y vim wget curl git net-tools
sudo: duf: command not found
[root@openeuler-lab ~]# sudo dnf install -y vim wget curl git net-tools
Last metadata expiration check: 0:06:49 ago on Sat 24 Oct 2026 04:36:30 PM CST.
Package vim-enhanced-2:9.0.2092-45.oe2403sp3.x86_64 is already installed.
Package wget-1.21.4-5.oe2403sp3.x86_64 is already installed.
Package curl-8.4.0-37.oe2403sp3.x86_64 is already installed.
Package git-2.43.0-13.oe2403sp3.x86_64 is already installed.
Package net-tools-2.10-5.oe2403sp3.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[root@openeuler-lab ~]#
```

报告图 64： 常用工具安装结果

```
[root@openeuler-lab ~]# sudo firewall-cmd --permanent --add-service=ssh
Warning: ALREADY_ENABLED: ssh
success
[root@openeuler-lab ~]# sudo firewall-cmd --reload
success
[root@openeuler-lab ~]#
```

报告图 66： SSH 规则已存在，重载成功

```
[root@openeuler-lab ~]# sudo systemctl enable --now sshd
[root@openeuler-lab ~]# sudo systemctl status sshd
● sshd.service - OpenSSH server daemon
     Loaded: loaded (/usr/lib/systemd/system/sshd.service; enabled; preset: enabled)
     Active: active (running) since Sat 2026-10-24 16:37:04 CST; 8min ago
       Docs: man:sshd(8)
             man:sshd_config(5)
   Main PID: 3828 (sshd)
      Tasks: 1 (limit: 21352)
     Memory: 1.2M ()
     CGroup: /system.slice/sshd.service
             └─3828 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Oct 24 16:37:04 openeuler-lab systemd[1]: Starting OpenSSH server daemon...
Oct 24 16:37:04 openeuler-lab (sshd)[3828]: sshd.service: Referenced but unset environment variable evaluates to an empty string: OPTIONS
Oct 24 16:37:04 openeuler-lab sshd[3828]: Server listening on 0.0.0.0 port 22.
Oct 24 16:37:04 openeuler-lab sshd[3828]: Server listening on :: port 22.
Oct 24 16:37:04 openeuler-lab systemd[1]: Started OpenSSH server daemon.
[root@openeuler-lab ~]# _
```

报告图 65： sshd 为 active (running)

虚拟机内已验证

工具可用、 SSH 启动、防火墙放行

主机连接条件

还需确认路由、 NAT 转发或桥接

服务状态与防火墙验证完成；主机到虚拟机的登录链路需另行验证

进程管理的核心认识与后续改进

实验内容	本次形成的认识
3.1—3.2 观察与控制	用 PID 、状态和 wait 理解进程生命周期
3.3—3.4 队列与调度	状态迁移与运行次序依赖正确的队列操作
3.5 日志与数据	在统一统计口径下比较受控实验
3.6 系统环境	用版本、网络和服务结果验证安装

后续改进　队列边界处理　多轮统计对照　虚拟机快照与连接验证

小组成员：陈心怡、杨骅、李金盘、李玥辰

用运行证据解释原理，用清晰口径约束结论